

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN II

InsightAI

*Hệ thống Chatbot RAG lai (Hybrid RAG)
cho phân tích tài liệu chuyên sâu*

Sinh viên thực hiện : Trần Chí Vũ

Mã số sinh viên : 20236060

Lớp : Công nghệ thông tin Việt-Pháp K68

Giảng viên hướng dẫn : Đỗ Bá Lâm

Học kỳ : 2025.2

Hà Nội, tháng 6 năm 2026

Mục lục

1	Mô tả bài toán	2
1.1	Bối cảnh và động lực	2
1.2	Phát biểu bài toán	2
1.3	Yêu cầu chức năng và phi chức năng	3
2	Cơ sở lý thuyết và kiến thức liên quan	3
2.1	Kiến trúc RAG tổng quát	3
2.2	Embedding và tìm kiếm ngữ nghĩa	3
2.3	Tìm kiếm từ khóa BM25	4
2.4	Hợp nhất kết quả bằng RRF	4
2.5	Tái xếp hạng bằng Cross-Encoder	4
2.6	HyDE – Hypothetical Document Embeddings	4
2.7	MMR và TextRank cho tóm tắt	5
3	Cách giải quyết	5
3.1	Tổng quan kiến trúc hệ thống	5
3.2	Giai đoạn 1 – Nạp và lập chỉ mục tài liệu	6
3.2.1	Trích xuất văn bản thông minh	6
3.2.2	Phân đoạn thích nghi (Adaptive Chunking)	6
3.3	Giai đoạn 2 – Định tuyến truy vấn động	7
3.4	Giai đoạn 3a – Pipeline tra cứu (Fact)	7
3.5	Giai đoạn 3b – Pipeline tóm tắt (Summary)	8
3.6	Giai đoạn 4 – Sinh câu trả lời và streaming	8
3.7	Cơ chế dự phòng nhiều nhà cung cấp LLM	9
3.8	Các quyết định thiết kế và đánh đổi	9
4	Kết quả	9
4.1	Sản phẩm đạt được	9
4.2	Các endpoint API chính	10
4.3	Đánh giá định tính	10
4.4	Ảnh chụp màn hình (minh họa)	11
5	Bài học và trải nghiệm thu được	12
5.1	Về kỹ thuật	13
5.2	Về kiến trúc phần mềm	13
5.3	Hạn chế và hướng phát triển	13
5.4	Kết luận	14

1. Mô tả bài toán

1.1. Bối cảnh và động lực

Sự bùng nổ của các mô hình ngôn ngữ lớn (Large Language Models – LLM) như GPT, Gemini hay Llama đã thay đổi cách con người tương tác với thông tin. Tuy nhiên, khi áp dụng trực tiếp các mô hình này vào việc trả lời câu hỏi dựa trên một tài liệu cụ thể, ta gặp ba hạn chế cốt lõi:

- **Ảo giác (hallucination):** LLM có xu hướng “bịa” ra thông tin nghe có vẻ hợp lý nhưng không có thật, đặc biệt với những câu hỏi về số liệu, định nghĩa hoặc sự kiện cụ thể trong tài liệu.
- **Kiến thức bị đóng băng:** Mô hình chỉ biết những gì có trong dữ liệu huấn luyện và không thể truy cập một tài liệu PDF, DOCX mới mà người dùng vừa tải lên.
- **Giới hạn ngữ cảnh (context window):** Không thể nhồi toàn bộ một bài báo khoa học hàng chục trang vào prompt; điều này vừa tốn chi phí, vừa làm giảm chất lượng do hiện tượng “lost in the middle”.

InsightAI ra đời để giải quyết các vấn đề trên. Đây là một chatbot full-stack cho phép người dùng tải lên tài liệu (PDF, DOCX, CSV, TXT) và đặt câu hỏi bằng tiếng Việt, hệ thống sẽ trả lời *có căn cứ* dựa trên đúng nội dung tài liệu kèm theo trích dẫn nguồn (citation). Kỹ thuật nền tảng được sử dụng là **Retrieval-Augmented Generation (RAG)** – sinh văn bản tăng cường bằng truy xuất.

1.2. Phát biểu bài toán

Cho một tập tài liệu $D = \{d_1, d_2, \dots, d_n\}$ do người dùng tải lên và một câu hỏi q bằng ngôn ngữ tự nhiên, hệ thống cần:

1. Trích xuất văn bản, phân đoạn (chunking) và lập chỉ mục (indexing) tài liệu thành một cấu trúc cho phép tìm kiếm hiệu quả.
2. Với mỗi câu hỏi q , truy xuất tập đoạn văn bản liên quan nhất $C_q \subseteq D$.
3. Sinh câu trả lời $a = \text{LLM}(q, C_q)$ chỉ dựa trên C_q , kèm danh sách nguồn trích dẫn.
4. Duy trì ngữ cảnh hội thoại để hiểu các câu hỏi nối tiếp (follow-up).

1.3. Yêu cầu chức năng và phi chức năng

Bảng 1: Tổng hợp yêu cầu của hệ thống InsightAI

Nhóm	Yêu cầu
Chức năng	Tải lên đa định dạng (PDF/DOCX/CSV/TXT); hỏi đáp grounded; trích dẫn nguồn; tóm tắt đa đoạn; bộ nhớ hội thoại theo phiên; streaming câu trả lời theo token.
Phi chức năng	Độ trễ thấp (xử lý bất đồng bộ); bảo mật/riêng tư (hỗ trợ LLM chạy cục bộ); độ chính xác truy xuất cao; khả năng mở rộng (kiến trúc tách lớp); giao diện đáp ứng (responsive, dark mode).

2. Cơ sở lý thuyết và kiến thức liên quan

2.1. Kiến trúc RAG tổng quát

RAG là một mẫu kiến trúc kết hợp hai thành phần: một *bộ truy xuất* (retriever) tìm các đoạn văn bản liên quan từ kho tri thức, và một *bộ sinh* (generator) là LLM tổng hợp câu trả lời từ những đoạn đó. Công thức tổng quát:

$$p(a \mid q) = \sum_{c \in C_q} \underbrace{p(c \mid q)}_{\text{truy xuất}} \cdot \underbrace{p(a \mid q, c)}_{\text{sinh}}$$

Ưu điểm của RAG là giảm ảo giác (câu trả lời phải bám vào tài liệu), cập nhật tri thức không cần huấn luyện lại, và cung cấp khả năng truy vết nguồn.

2.2. Embedding và tìm kiếm ngữ nghĩa

Embedding là phép ánh xạ một đoạn văn bản thành một vector số thực $v \in \mathbb{R}^d$ sao cho các đoạn có ý nghĩa gần nhau nằm gần nhau trong không gian vector. InsightAI sử dụng mô hình BAAI/bge-small-en-v1.5 (384 chiều) qua thư viện HuggingFace. Độ tương đồng giữa hai đoạn được đo bằng *cosine similarity*:

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

Tập vector được lưu trong **FAISS** (Facebook AI Similarity Search) – một thư viện cho phép tìm k vector gần nhất (k-NN) cực nhanh trên hàng triệu vector.

2.3. Tìm kiếm từ khóa BM25

Tìm kiếm ngữ nghĩa giỏi nắm bắt ý nghĩa nhưng thường *bỏ sót* các thuật ngữ chính xác (tên riêng, mã sản phẩm, ký hiệu hiếm). Để bù lại, InsightAI dùng thêm **BM25** – một hàm xếp hạng dựa trên tần suất từ khóa. Điểm BM25 của tài liệu d với truy vấn q :

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}$$

trong đó $f(t, d)$ là tần suất từ t trong d , $|d|$ là độ dài tài liệu, avgdl là độ dài trung bình, còn k_1, b là siêu tham số. Việc kết hợp BM25 (thưa, theo từ khóa) với tìm kiếm dense (đặc, theo ngữ nghĩa) gọi là **Hybrid Search**, giải quyết bài toán “vocabulary mismatch”.

2.4. Hợp nhất kết quả bằng RRF

Hai bộ truy xuất trả về hai danh sách xếp hạng khác nhau. InsightAI hợp nhất chúng bằng **Reciprocal Rank Fusion (RRF)** – một phương pháp không cần chuẩn hóa điểm số, chỉ dựa trên thứ hạng:

$$\text{RRF}(d) = \sum_i \frac{1}{k_{\text{rrf}} + \text{rank}_i(d)}$$

với $\text{rank}_i(d)$ là vị trí của tài liệu d trong danh sách thứ i và $k_{\text{rrf}} = 60$ (giá trị kinh điển). RRF mạnh ở chỗ tài liệu xuất hiện ở thứ hạng cao trong *nhiều* danh sách sẽ được ưu tiên.

2.5. Tái xếp hạng bằng Cross-Encoder

Sau khi hợp nhất, hệ thống có một tập ứng viên. Để chọn ra những đoạn tốt nhất, InsightAI dùng một **Cross-Encoder** (cross-encoder/ms-marco-MiniLM-L-6-v2). Khác với bi-encoder (mã hóa truy vấn và tài liệu độc lập), cross-encoder đưa *cặp* (truy vấn, đoạn) vào cùng một lần forward nên đánh giá độ liên quan chính xác hơn nhiều, đổi lại chậm hơn – vì vậy chỉ dùng ở bước cuối trên một tập nhỏ ứng viên (mẫu thiết kế *retrieve-then-rerank*).

2.6. HyDE – Hypothetical Document Embeddings

Với tài liệu học thuật, câu hỏi của người dùng và văn phong bài báo thường khác nhau về từ vựng. **HyDE** khắc phục bằng cách: yêu cầu LLM sinh một câu trả lời *giả định* cho câu hỏi, rồi embed câu trả lời giả định đó thay vì câu hỏi gốc. Vì câu trả lời giả định gần với văn phong bài báo hơn, độ trùng khớp ngữ nghĩa tăng lên đáng kể.

2.7. MMR và TextRank cho tóm tắt

Khi câu hỏi mang tính tóm tắt, mục tiêu không phải độ chính xác điểm mà là *độ bao phủ* và *tính đa dạng*. InsightAI dùng:

- **Maximal Marginal Relevance (MMR)**: chọn các đoạn vừa liên quan truy vấn vừa khác biệt nhau, tránh trùng lặp:

$$\text{MMR} = \arg \max_{d_i \in C \setminus S} \left[\lambda \text{sim}(d_i, q) - (1 - \lambda) \max_{d_j \in S} \text{sim}(d_i, d_j) \right]$$

- **TextRank**: một biến thể PageRank áp dụng trên đồ thị câu (cạnh là độ tương đồng), xếp hạng các câu quan trọng nhất để chắt lọc ngữ cảnh trước khi đưa cho LLM.

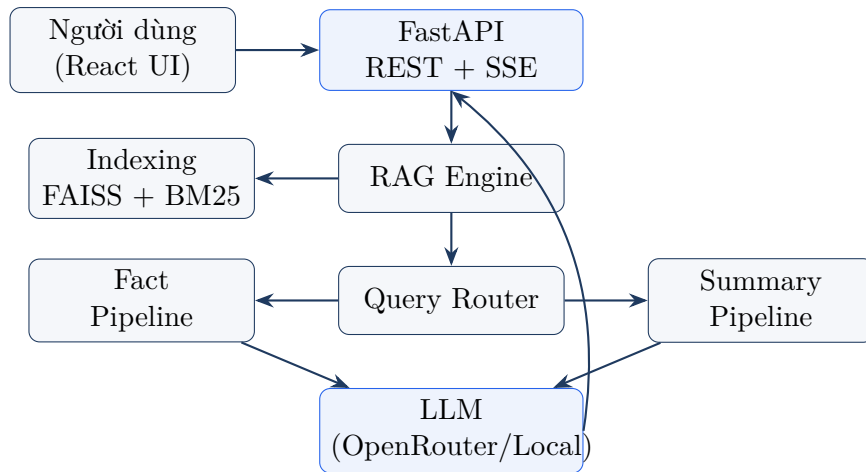
3. Cách giải quyết

3.1. Tổng quan kiến trúc hệ thống

InsightAI được xây dựng theo kiến trúc tách lớp rõ ràng giữa backend và frontend.

Bảng 2: Công nghệ sử dụng

Tầng	Công nghệ
Frontend	React 18, TypeScript, Vite, Tailwind CSS, react-markdown + KaTeX (render công thức), giao diện dark mode.
Backend	Python 3.10, FastAPI (bất đồng bộ), Uvicorn.
RAG & AI	sentence-transformers, FAISS, rank-bm25, CrossEncoder, LangChain.
Trích xuất	PyMuPDF (PDF có nhận diện tiêu đề mục), docx2txt, csv.
LLM	OpenRouter (chính), LM Studio (cục bộ), Google Gemini (tùy chọn).
Bộ nhớ	In-memory hoặc Upstash Redis (theo phiên, có TTL).
Triển khai	Docker & Docker Compose.



Hình 1: Sơ đồ kiến trúc tổng quan của InsightAI

3.2. Giai đoạn 1 – Nạp và lập chỉ mục tài liệu

3.2.1. Trích xuất văn bản thông minh

Với PDF, hệ thống không chỉ lấy text thô mà còn *nhận diện tiêu đề mục* bằng heuristic cỡ chữ: thu thập toàn bộ kích thước font, lấy cỡ phổ biến nhất làm cỡ thân bài, mọi dòng có font lớn hơn 15% và khớp danh sách tên mục học thuật (*Abstract, Introduction, Methodology, ...*) được gán làm `section_title`. Nhờ đó mỗi đoạn văn bản đều mang ngữ cảnh “nó thuộc mục nào”, phục vụ trích dẫn chính xác.

3.2.2. Phân đoạn thích nghi (Adaptive Chunking)

Hệ thống tự động phát hiện loại tài liệu (`academic_paper`, `legal`, `general`) qua từ khóa, rồi chọn chiến lược chunking phù hợp:

- **Bài báo học thuật:** chunking theo ranh giới mục (*section-aware*) – nhanh, vì mục đã cung cấp sẵn ngữ cảnh ngữ nghĩa.
- **Tài liệu chung/pháp lý:** dùng `SemanticChunker` – cắt theo “điểm chuyển ý” dựa trên embedding, cho chất lượng cao hơn.
- **Chế độ fixed:** cắt theo kích thước cố định (mặc định 600 ký tự, chồng lấn 120) khi cần tốc độ và sự ổn định.

Sau khi chunk, hệ thống xây dựng song song **ba cấu trúc**: chỉ mục FAISS (dense), chỉ mục BM25 (keyword), và các **block** lớn (cửa sổ trượt 15 chunk, chồng lấn 3) phục vụ truy xuất tóm tắt theo mẫu *small-to-big*.

Listing 1: Xây dựng chỉ mục lai trong `indexing.py`

```
1 # ỰT phát ệhìn ại loi tài ệliu -> ọchn ếchìn ượlc chunking
```

```

2 engine.doc_type = detect_doc_type(docs)
3
4 if chunking_mode == "semantic" and engine.doc_type == "academic_paper":
5     parent_docs = section_aware_split(docs)          # nhanh
6 elif chunking_mode == "semantic":
7     parent_docs = engine.parent_splitter.split_documents(docs) #
8     SemanticChunker
9
10 # iCh umeric dense (FAISS) + keyword (BM25)
11 engine.vectorstore = FAISS.from_documents([parent_docs[0]], engine.
12     embeddings)
13 engine.bm25_corpus = [tokenize(doc.page_content) for doc in engine.
14     all_chunks]
15 engine.bm25_index = BM25Okapi(engine.bm25_corpus)
16 create_blocks(engine) # block ở ln cho pipeline tóm tắt

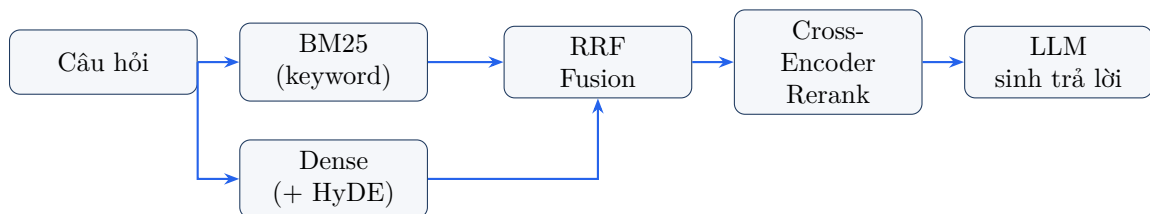
```

3.3. Giai đoạn 2 – Định tuyến truy vấn động

Mỗi câu hỏi đi qua một **Query Router** thực hiện ba bước:

1. **Viết lại theo lịch sử:** nếu có `session_id`, câu hỏi nối tiếp (“thế còn phần thực nghiệm?”) được LLM viết lại thành câu hỏi độc lập dựa trên 5 lượt hội thoại gần nhất.
2. **Phân loại ý định:** xác định câu hỏi thuộc loại **fact** (tra cứu cụ thể) hay **summary** (tóm tắt, ý chính) – trước hết bằng từ khóa, sau đó bằng LLM nếu cần.
3. **Định tuyến:** chuyển câu hỏi tới pipeline tương ứng.

3.4. Giai đoạn 3a – Pipeline tra cứu (Fact)



Hình 2: Pipeline tra cứu chính xác cho câu hỏi dạng fact

Câu hỏi được truy xuất song song bằng BM25 và dense (kèm HyDE nếu là bài báo học thuật), hợp nhất bằng RRF, rồi tái xếp hạng bằng cross-encoder để lấy ra k đoạn tốt nhất đưa vào LLM. Toàn bộ pipeline thể hiện cô đọng trong đoạn mã sau:

Listing 2: Pipeline fact trong fact_service.py

```

1 def run_fact_pipeline(self, user_query, k):
2     bm25_results = retrieval.bm25_retrieve(self.engine, user_query, k=k
3     *5)
4     use_hyde = self.engine.doc_type == "academic_paper"
5     dense_results = (retrieval.dense_retrieve_with_hyde if use_hyde
6                     else retrieval.dense_retrieve)(self.engine,
7     user_query, k=k*5)
8     fused = retrieval.rrf_fusion([bm25_results, dense_results], k=k*3)
9     reranked = retrieval.rerank(self.engine, user_query, fused, top_k=k)
10    return bm25_results, dense_results, fused, reranked

```

3.5. Giai đoạn 3b – Pipeline tóm tắt (Summary)

Với câu hỏi tóm tắt, hệ thống truy xuất các **block lớn** liên quan, trả chúng thành các chunk con, dùng **MMR** chọn tập chunk đa dạng, rồi **TextRank + MMR ở cấp câu** để chắt lọc những câu cốt lõi nhất. Ngữ cảnh đã tinh gọn này mới được đưa cho LLM viết bản tóm tắt mạch lạc, tránh hiện tượng “lost in the middle”.

3.6. Giai đoạn 4 – Sinh câu trả lời và streaming

LLM được điều khiển bằng một system prompt thiết kế kỹ (vai trò trợ lý đọc hiểu bài báo), ràng buộc *chỉ dùng thông tin trong ngữ cảnh*, không bịa, và nếu thiếu dữ liệu phải nói rõ “Không có trong tài liệu”. Câu trả lời được trả về kèm danh sách citation (tên tài liệu, trang, mục, đoạn trích).

Hệ thống hỗ trợ **streaming** qua Server-Sent Events (SSE): backend gọi `next()` trên generator đồng bộ trong thread pool (`run_in_executor`) để event loop vẫn rảnh đẩy từng token về client, tạo trải nghiệm “gõ chữ” mượt mà.

Listing 3: Endpoint streaming SSE trong chat.py

```

1 async def event_generator():
2     loop = asyncio.get_running_loop()
3     sync_gen = rag_engine.stream_query(request.query, session_id=request
4     .session_id)
5     def get_next():
6         try: return next(sync_gen)
7         except StopIteration: return _DONE
8     try:
9         while True:
10            item = await loop.run_in_executor(None, get_next)
11            if item is _DONE: break
12            yield item # data: {"type": "token", "content": "..."}
13    except Exception as e:

```

```
13 yield f"data: {json.dumps({'type':'error','content':str(e)})}\n\n"
```

3.7. Cơ chế dự phòng nhiều nhà cung cấp LLM

Để cân bằng giữa *riêng tư* và *chất lượng*, InsightAI hỗ trợ nhiều nhà cung cấp với thứ tự ưu tiên: OpenRouter (đám mây, chính) → LM Studio (mô hình cục bộ, riêng tư tuyệt đối). Google Gemini cũng được cấu hình sẵn và có thể kích hoạt dễ dàng. Khi một nhà cung cấp lỗi hoặc bị rate-limit, hệ thống tự chuyển sang lựa chọn kế tiếp, đảm bảo dịch vụ luôn sẵn sàng.

3.8. Các quyết định thiết kế và đánh đổi

Bảng 3: Một số quyết định kiến trúc và lý do

Quyết định	Lý do / Đánh đổi
FastAPI thay vì Flask	RAG là tác vụ I/O-bound; <code>async/await</code> cho phép xử lý nhiều yêu cầu đồng thời mà không chặn event loop.
Hybrid (BM25 + Dense)	Dense giải ngữ nghĩa nhưng bỏ sót từ khóa; BM25 ngược lại. Kết hợp bằng RRF cho recall và precision cao.
LLM cục bộ + đám mây	Cục bộ: miễn phí, dữ liệu không rời máy (riêng tư); đám mây: suy luận mạnh, không cần hạ tầng. Người dùng được chọn.
Semantic chunking	Cắt theo “điểm chuyển ý” thay vì cắt cứng giữa câu, giúp mỗi đoạn mạch lạc hơn, tăng độ chính xác truy xuất.
Block-based summary	Lấy block lớn để giữ ngữ cảnh, lọc lại bằng MMR để LLM chỉ nhận thông tin đa dạng, đậm đặc, tránh “lost in the middle”.

4. Kết quả

4.1. Sản phẩm đạt được

Hệ thống InsightAI hoàn chỉnh đã được triển khai thành công với đầy đủ các chức năng đề ra:

- Giao diện chat hiện đại, hỗ trợ render Markdown và công thức toán (KaTeX), hiển thị nguồn trích dẫn cho từng câu trả lời.
- Tải lên đồng thời nhiều tệp PDF/DOCX/CSV/TXT với thanh tiến trình indexing thời gian thực qua endpoint `/progress`.

- Hỏi đáp tiếng Việt grounded với hai chế độ tra cứu và tóm tắt tự động định tuyến.
- Bộ nhớ hội thoại theo phiên cho phép hỏi nối tiếp tự nhiên.
- Câu trả lời streaming theo token tạo trải nghiệm mượt mà.
- Triển khai một lệnh bằng Docker Compose (`docker compose up`).

4.2. Các endpoint API chính

Bảng 4: Các endpoint REST của backend

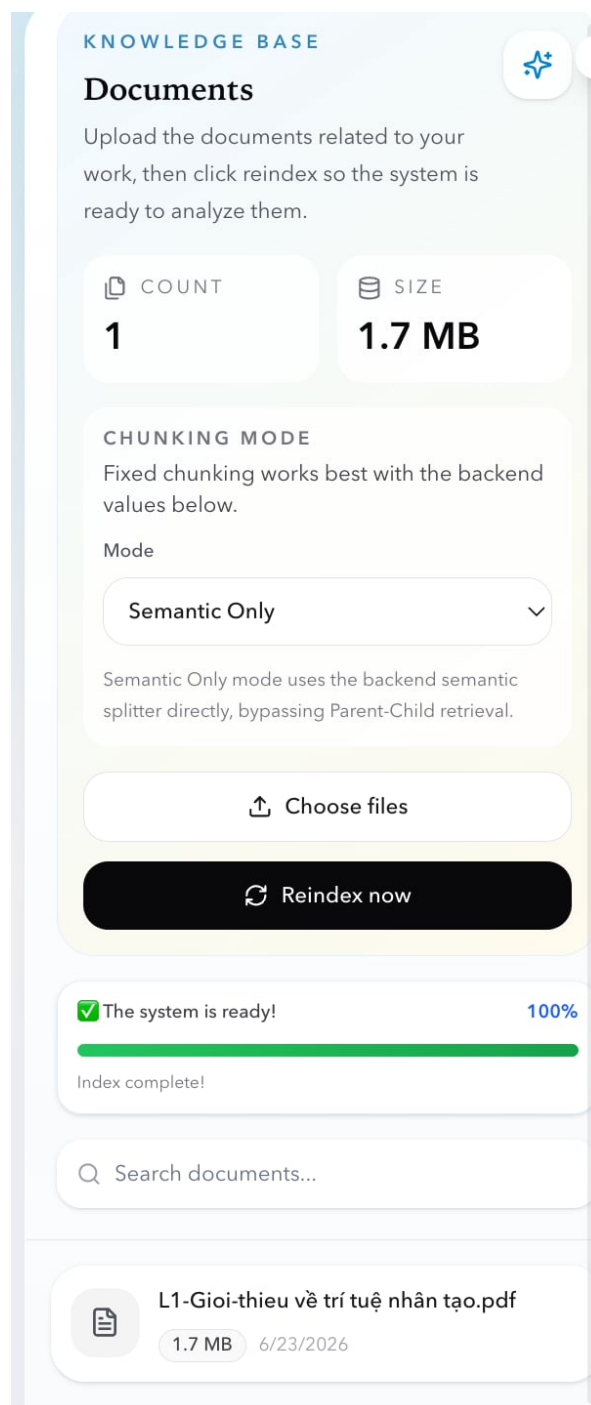
Endpoint	Chức năng
POST /upload	Tải tệp lên và index nền (background task).
GET /progress	Trả về trạng thái và % tiến trình indexing.
POST /queryHybrid	Hỏi đáp, trả về {answer, sources}.
POST /queryHybrid/stream	Hỏi đáp dạng streaming SSE.
POST /reset	Xóa toàn bộ chỉ mục.
GET /status	Kiểm tra sức khỏe và số vector.

4.3. Đánh giá định tính

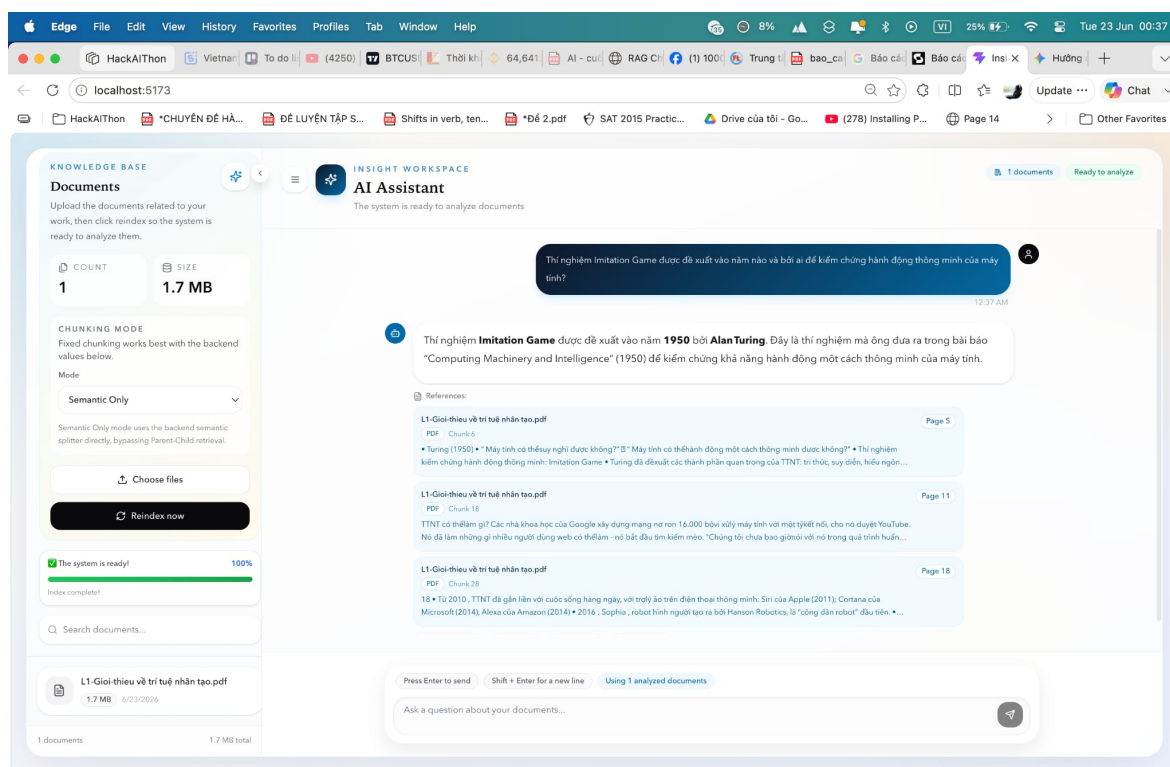
Qua thử nghiệm với các bài báo khoa học (định dạng arXiv) và CV, hệ thống cho thấy:

- **Câu hỏi fact** (“Mô hình đạt độ chính xác bao nhiêu trên tập test?”): nhờ hybrid + rerank, hệ thống truy xuất đúng đoạn chứa số liệu và trả lời chính xác, kèm trích dẫn trang/mục.
- **Câu hỏi summary** (“Tóm tắt ý chính của bài báo”): pipeline block + MMR + TextRank cho bản tóm tắt bao quát cả các mục Introduction, Method, Results mà không lặp ý.
- **Chống ảo giác**: với câu hỏi nằm ngoài tài liệu, hệ thống tuân thủ ràng buộc và trả lời “Không có trong tài liệu” thay vì bịa.
- **Hội thoại nối tiếp**: cơ chế viết lại truy vấn giúp hiểu đúng các câu hỏi rút gọn dựa trên ngữ cảnh trước đó.

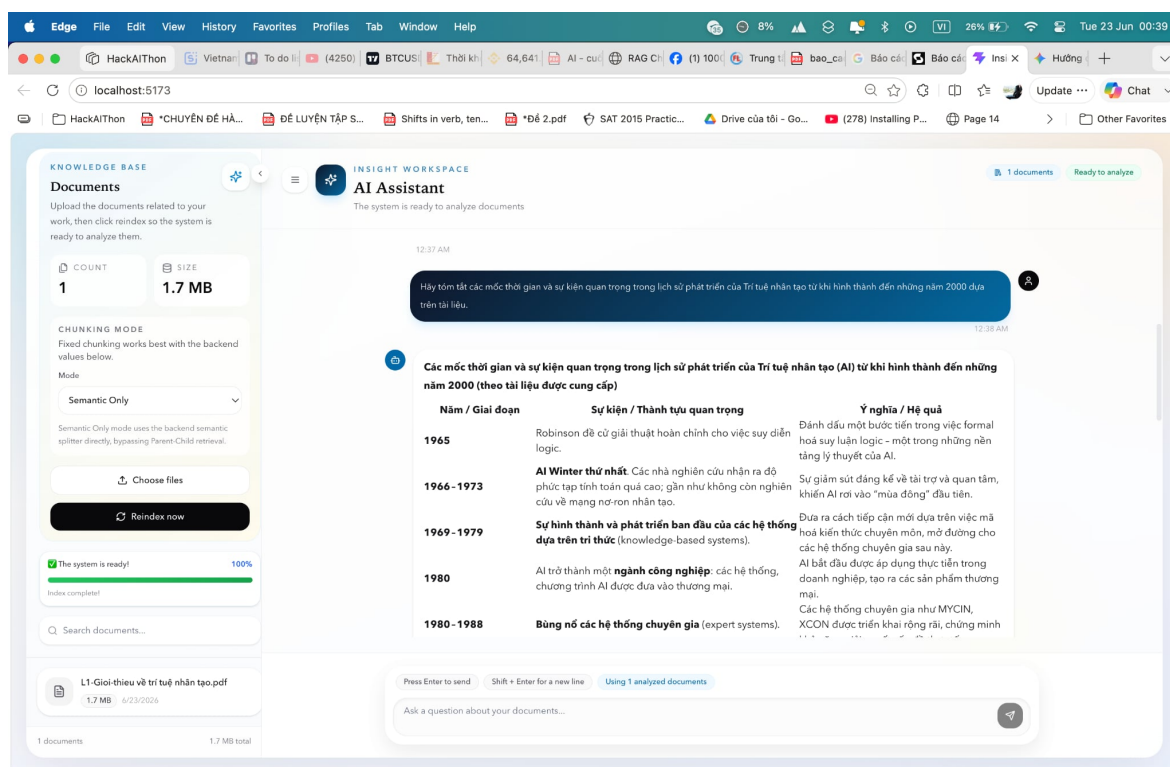
4.4. Ảnh chụp màn hình (minh họa)



Hình 3: Màn hình tải tài liệu và thanh tiến trình indexing



Hình 4: Câu hỏi fact và câu trả lời kèm citation



Hình 5: Câu hỏi tóm tắt và bản tóm tắt đa đoạn

5. Bài học và trải nghiệm thu được

5.1. Về kỹ thuật

- **RAG không chỉ là “embed rồi tìm”**: chất lượng phụ thuộc rất lớn vào từng mắt xích – chunking, hybrid retrieval, fusion, reranking. Việc thêm cross-encoder rerank và RRF nâng độ liên quan của ngữ cảnh lên rõ rệt so với RAG cơ bản.
- **Chunking là gốc rễ chất lượng**: cắt sai làm vỡ ngữ cảnh và kéo theo toàn bộ pipeline kém đi. Chiến lược thích nghi theo loại tài liệu là một bài học quan trọng về việc “không có giải pháp một-cho-tất-cả”.
- **Tách câu hỏi fact và summary**: mỗi loại cần một chiến lược truy xuất khác nhau (precision vs. coverage). Định tuyến động giúp hệ thống linh hoạt mà không đánh đổi chất lượng.
- **Lập trình bất đồng bộ**: hiểu sâu hơn về `async/await`, `run_in_executor` và SSE khi triển khai streaming – cầu nối giữa code đồng bộ (thư viện ML) và server bất đồng bộ.
- **Thiết kế prompt và ràng buộc**: một system prompt chặt chẽ là tuyến phòng thủ cuối cùng chống ảo giác; ràng buộc “chỉ dùng ngữ cảnh” có tác động lớn đến độ tin cậy.

5.2. Về kiến trúc phần mềm

Việc tách các dịch vụ RAG (`indexing`, `retrieval`, `fact_service`, `summarization_service`, `generator`) thành các module độc lập giúp code dễ đọc, dễ kiểm thử và dễ thay thế từng thành phần (ví dụ đổi vector store hay nhà cung cấp LLM mà không ảnh hưởng phần còn lại). Mẫu *dependency injection* của FastAPI và lớp `ConversationStore` có khả năng chuyển từ in-memory sang Redis cho thấy giá trị của việc thiết kế hướng tới khả năng mở rộng ngay từ đầu.

5.3. Hạn chế và hướng phát triển

- **Chưa có đánh giá định lượng**: cần xây dựng bộ test với các metric như Recall@k, MRR, hoặc dùng framework RAGAS để đo faithfulness và relevancy.
- **Embedding tiếng Anh**: mô hình BAAI/bge-small-en-v1.5 tối ưu cho tiếng Anh; với tài liệu tiếng Việt nên thử các mô hình đa ngôn ngữ.
- **Mở rộng**: hỗ trợ truy xuất đa phương thức (hình, bảng), bộ nhớ hội thoại bền vững, và caching kết quả truy xuất để giảm độ trễ.

5.4. Kết luận

Đồ án InsightAI đã hiện thực hóa thành công một hệ thống chatbot RAG lai hoàn chỉnh, giải quyết được bài toán hỏi đáp grounded trên tài liệu do người dùng tải lên. Qua quá trình thực hiện, em không chỉ nắm vững các kỹ thuật cốt lõi của RAG hiện đại (hybrid search, reranking, HyDE, MMR, định tuyến truy vấn) mà còn rèn luyện kỹ năng thiết kế kiến trúc phần mềm tách lớp, lập trình bất đồng bộ và triển khai một sản phẩm full-stack từ đầu đến cuối. Đây là nền tảng vững chắc cho các nghiên cứu và ứng dụng AI thực tế trong tương lai.

Tài liệu

- [1] P. Lewis, E. Perez, A. Piktus, *et al.* “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [2] BAAI. “bge-small-en-v1.5 Embedding Model,” Hugging Face Model Hub, <https://huggingface.co/BAAI/bge-small-en-v1.5>.
- [3] Cross-Encoder. “ms-marco-MiniLM-L-6-v2,” Hugging Face Model Hub, <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2>.
- [4] J. Johnson, M. Douze, H. Jégou. “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, 2019.
- [5] S. Robertson, H. Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, 2009.
- [6] G. V. Cormack, C. L. A. Clarke, S. Büttcher. “Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods,” *SIGIR*, 2009.
- [7] L. Gao, X. Ma, J. Lin, J. Callan. “Precise Zero-Shot Dense Retrieval without Relevance Labels,” *ACL*, 2023.
- [8] LangChain AI. “LangChain Documentation,” <https://python.langchain.com/docs>.
- [9] S. Ramirez. “FastAPI Documentation,” <https://fastapi.tiangolo.com>.